

UNIVERSIDAD DA VINCI DE GUATEMALA
FACULTAD DE INGENIERÍA, INDUSTRIA Y TECNOLOGÍA
CARRERA: LICENCIATURA EN INGENIERÍA EN SISTEMAS
CURSO: ESTRUCTURA DE DATOS



TDA COLA

Evaluación parcial

Jimmy Javier Martínez Sipac
Carnet 202302745

Guatemala, 29 de abril de 2024

Introducción

El desarrollo de la segunda evaluación parcial implica presentar la especificación e implementación del TDA Cola. Un Tipo de Dato Abstracto (TDA) se define como un modelo que especifica los valores y las operaciones que se pueden realizar sobre ellos, con la intención de que el usuario no necesite conocer los detalles internos de la representación o la implementación de las operaciones.

En este ensayo, se explorará la implementación del TDA de colas aplicando el método FIFO (Primero en entrar, primero en ser atendido) en Java, examinando sus componentes clave, operaciones básicas y consideraciones de diseño. A través de esta exploración, se busca resaltar la importancia y utilidad del TDA de colas en el desarrollo de software, su papel en la simplificación y optimización de tareas relacionadas con operaciones de fracciones.

Para ilustrar el concepto de una cola FIFO, consideremos el ejemplo de una cola en un banco. Cuando los clientes llegan al banco, se forman en una fila (cola). El primer cliente en llegar es el primero en ser atendido por el cajero, siguiendo el principio de "primero en entrar, primero en ser atendido". Este proceso refleja el comportamiento de una cola FIFO, donde los elementos se procesan en el orden en que llegaron.

En los anexos, se proporcionan referencias a los repositorios donde se implementó el TDA en lenguaje de código Java, así como un plan de prueba para la implementación.

TDA Cola Medieval

¿Qué es una cola en estructura de datos?



Esquema de cola [Esquema]. (Masai School, 2024)

Una cola en el contexto de las estructuras de datos representa una secuencia de elementos donde el primer elemento ingresado es también el primero en ser eliminado, siguiendo el principio conocido como FIFO (First-In, First-Out).

En este tipo de estructura, los elementos se añaden al final de la cola mediante operaciones como "encolar" o "push", y se eliminan del principio de la cola mediante operaciones como "desencolar" o "pop". Este comportamiento es similar al de una fila de personas esperando en un mostrador de atención, donde aquellos que llegan primero son los primeros en ser atendidos.

Las colas son especialmente útiles en casos donde se requiere preservar el orden de procesamiento, garantizando que los elementos se manejen en el mismo orden en el que fueron ingresados. Este concepto es fundamental en numerosas aplicaciones informáticas, como la administración de tareas en sistemas operativos, la comunicación entre procesos, la impresión de documentos en impresoras, y la gestión de solicitudes en servidores web, entre otras.

Tipos de colas

Cola Simple

Los elementos se eliminan en el mismo orden en que se agregaron.

Cola con Prioridad

Los elementos se procesan según su prioridad, no necesariamente en el orden de llegada.

Cola de Doble Extremo (Deque)

Permite agregar y eliminar elementos tanto al principio como al final de la cola.

Cola Circular

Los elementos se organizan en una estructura circular, permitiendo un uso eficiente del espacio.

Cola Concurrente

Diseñada para el acceso seguro desde múltiples hilos de ejecución.

Cola de Búfer (Buffer Queue)

Utilizada para almacenar temporalmente datos entre procesos o dispositivos.

Operaciones

Con las colas se pueden realizar las siguientes operaciones:

Encolar o añadir

Para agregar un nuevo elemento a la cola medieval, se utiliza el método añadir. Este método toma dos parámetros: el nombre del elemento a añadir y un booleano que indica si el elemento es un noble (true) o un plebeyo (false). Dependiendo de este último parámetro, el elemento se agrega a la cola correspondiente de nobles o plebeyos.

Obtener primer elemento

Se utiliza para obtener el primer elemento de la cola. Si hay elementos presentes en la cola, este método devuelve el nombre del primer elemento en la cola; de lo contrario, devuelve el primer elemento.

Desencolar o atender

Este método elimina el elemento de la cola, si hay elementos presentes; de lo contrario, elimina el primer elemento. De esta manera, se sigue el principio de "primero en entrar, primero en ser atendido" para ambos estamentos

Contar elementos

Para conocer el número total de elementos presentes en la cola, se utiliza este método. Este método devuelve el tamaño de la cola, es decir, cuántos elementos están esperando su turno de atención.

Verificar si está vacía

Se utiliza para verificar si la cola está vacía. Retorna true si la cola está vacía, indicando que no hay ningún elemento esperando ser atendido. En caso contrario, retorna false.

Especificación formal del TDA

Nombre: Fracción

1. Descripción del TDA:

El TDA Cola Medieval representa una estructura de datos que organiza elementos en dos estamentos: nobles y plebeyos, siguiendo el principio de "primero en entrar, primero en ser atendido". Permite realizar operaciones básicas como añadir un nuevo elemento, consultar el primer elemento, quitar el primer elemento, así como obtener el número de nobles y plebeyos en la cola y verificar si la cola está vacía

2. Invariante de TDA:

Para que el TDA Cola Medieval sea válido, debe cumplir con las siguientes invariantes:

2.1 Ordenamiento de las Colas

Las colas de nobles y plebeyos deben ser colecciones ordenadas donde el primer elemento agregado es el primero en ser atendido.

2.2 Mantenimiento del Tamaño

El tamaño de las colas de nobles y plebeyos debe mantenerse actualizado de acuerdo con las operaciones de añadir y quitar elementos.

2.3 Ausencia de Elementos Nulos

Las colas de nobles y plebeyos no deben contener elementos nulos.

2.4 Ausencia de Elementos Nulos

El tamaño de las colas de nobles y plebeyos no puede ser negativo.

3. Operaciones

Añadir

ConsultarPrimero

QuitarPrimero

ConsultarNumeroNobles

ConsultarNumeroPlebeyos

EsVacía

4. Definición de las operaciones

Referencias

Pandey, K. (2024, 19 abril). *Applications of Queue Explained*. Masai School.

<https://www.masaischool.com/blog/applications-of-queue/>

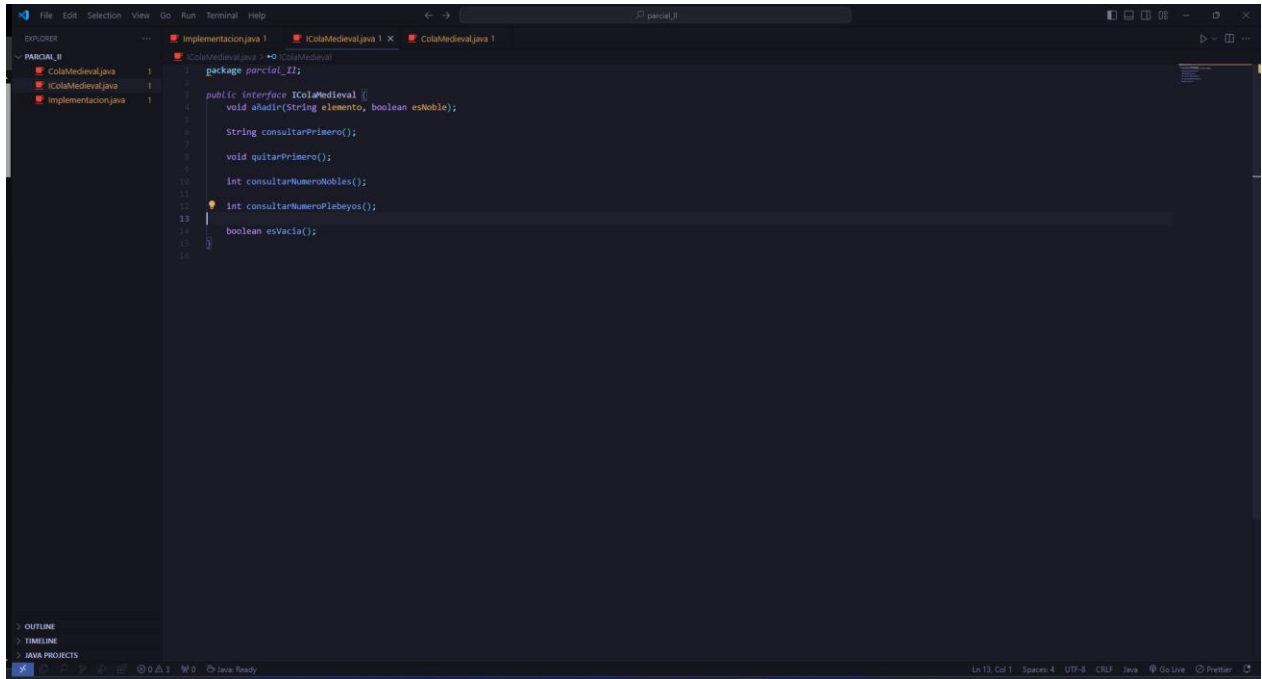
¿Qué es un Queue? - NI. (2017, 10 noviembre).

<https://knowledge.ni.com/KnowledgeArticleDetails?id=kA00Z000000P7OfSAK&l=es-ES>

Anexos

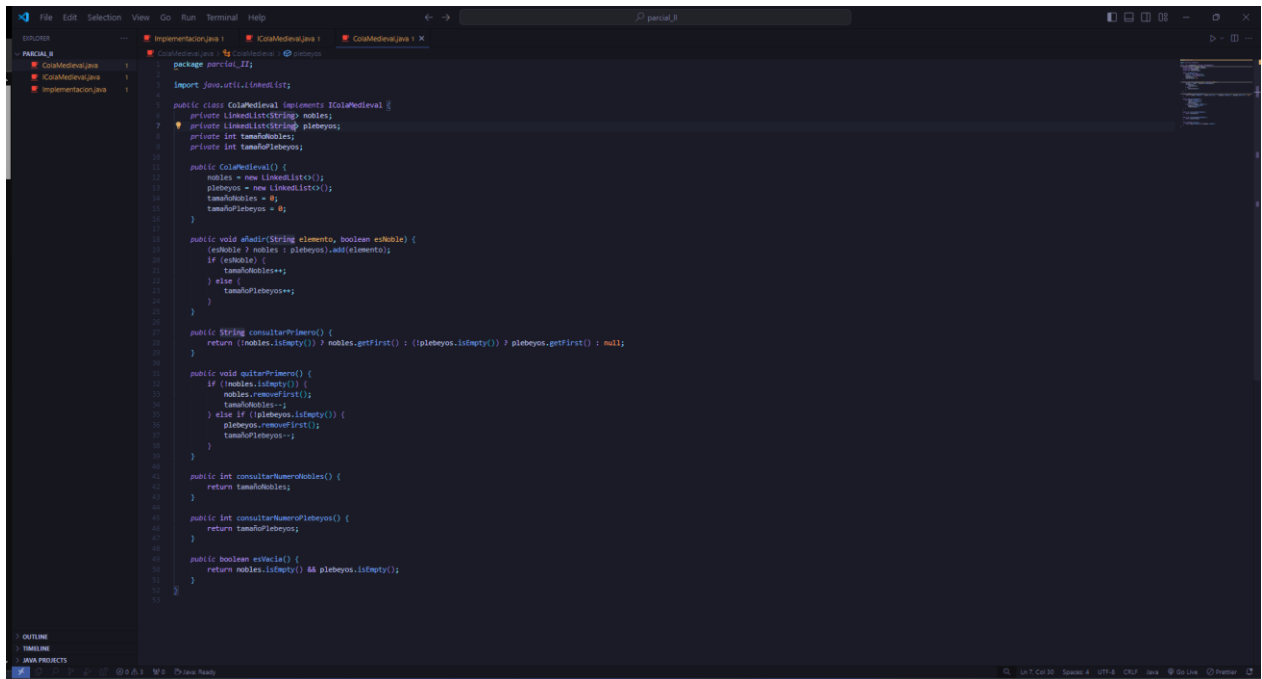
Programa fuente: Clase que implementa en Java el TDA Cola Medieval

https://github.com/MegaGone/udv/blob/develop/estructuras/parcial_II/ColaMedieval.java



This screenshot shows the 'ColaMedieval.java' file in an IDE. The file defines a package 'parcial_II' and a public interface 'IColaMedieval'. The interface includes methods for adding and removing elements, and querying the number of nobles and plebeians in the queue.

```
1 package parcial_II;
2
3 public interface IColaMedieval {
4     void aadir(String elemento, boolean esNoble);
5     String consultarPrimer();
6     void quitarPrimer();
7     int consultarNumeroNobles();
8     int consultarNumeroPlebeyos();
9     boolean esVacia();
10 }
```



This screenshot shows the 'ColaMedieval.java' file in an IDE, displaying the implementation of the 'IColaMedieval' interface. The class 'ColaMedieval' implements the interface methods using two linked lists to store nobles and plebeians.

```
1 package parcial_II;
2
3 import java.util.LinkedList;
4
5 public class ColaMedieval implements IColaMedieval {
6     private LinkedList<String> nobles;
7     private LinkedList<String> plebeyos;
8     private int tamañoNobles;
9     private int tamañoPlebeyos;
10
11     public ColaMedieval() {
12         nobles = new LinkedList<>();
13         plebeyos = new LinkedList<>();
14         tamañoNobles = 0;
15         tamañoPlebeyos = 0;
16     }
17
18     public void aadir(String elemento, boolean esNoble) {
19         (esNoble ? nobles : plebeyos).add(elemento);
20         if (esNoble) {
21             tamañoNobles++;
22         } else {
23             tamañoPlebeyos++;
24         }
25     }
26
27     public String consultarPrimer() {
28         return (nobles.isEmpty() ? nobles.getFirst() : (plebeyos.isEmpty() ? plebeyos.getFirst() : null));
29     }
30
31     public void quitarPrimer() {
32         if (nobles.isEmpty()) {
33             nobles.removeFirst();
34             tamañoNobles--;
35         } else if (plebeyos.isEmpty()) {
36             plebeyos.removeFirst();
37             tamañoPlebeyos--;
38         }
39     }
40
41     public int consultarNumeroNobles() {
42         return tamañoNobles;
43     }
44
45     public int consultarNumeroPlebeyos() {
46         return tamañoPlebeyos;
47     }
48
49     public boolean esVacia() {
50         return nobles.isEmpty() && plebeyos.isEmpty();
51     }
52 }
```

The screenshot displays an IDE window titled "personal_K". The main editor shows the source code for "ColeMedieval.java". The code defines a class "Implementation" with a static method "main" that interacts with a user via a scanner. It prompts for a name, checks if it's noble or commoner, and then asks for a number of items (weapons, armor, horses) based on the role.

```
// ColeMedieval.java //  
// Zilly Martinez - Carnet No. 280262745  
// Repo : https://github.com/ZillyMartinez/personal_k/tree/main/src/estructuras/parcial_22/Implementacion_Cole_Medieval  
  
import java.util.Scanner;  
  
public class Implementation {  
  
    Run@Debug  
    public static void main(String[] args) {  
        ColeMedieval cola = new ColeMedieval();  
  
        solicitarPersonajes(cola);  
  
        System.out.println("=====*****w=====");  
        System.out.println("EL NUMERO EN LA COLA ES: " + cola.consultarPrimeros());  
        System.out.println("NUMERO DE ARMOR EN LA COLA " + "*" + cola.consultarArmor() + "*");  
        cola.quitarArmor();  
        System.out.println("EL NUMERO EN LA COLA ACTUALIZADA ES: " + cola.consultarArmor());  
        System.out.println("NUMERO DE NOBLES EN LA COLA " + * + cola.consultarNumerosNobles());  
        System.out.println("NUMERO DE PUEBLOS EN LA COLA: " + cola.consultarNumerosPueblos());  
        System.out.println("ESTA VAZIA LA COLA? " + cola.vazia());  
        System.out.println("*****w=====");  
    }  
  
    private static void solicitarPersonajes(ColeMedieval cola) {  
        Scanner scanner = new Scanner(System.in);  
  
        for (int i = 0; i < 5; i++) {  
            System.out.println("Ingrese el nombre del personaje " + i + 1 + ":");  
            String nombre = scanner.nextLine();  
            while (!checkNombreValido(nombre)) {  
                System.out.println("  
                *El nombre del personaje debe ser una cadena de caracteres. Por favor, ingrese un nombre válido!");  
                nombre = scanner.nextLine();  
            }  
  
            System.out.println("Es noble (S/N)?");  
            String respuesta = scanner.nextLine().toLowerCase();  
            while (!esRespuestaValida(respuesta)) {  
                System.out.println("Por favor, responda 'S' o 'N'.");  
                respuesta = scanner.nextLine().toLowerCase();  
            }  
            boolean esNoble = respuesta.equals(Objects.toString("s"));  
            cola.agadir(nombre, esNoble);  
  
            scanner.close();  
        }  
  
        private static boolean esNombreValido(String nombre) {  
            return !nombre.isEmpty();  
        }  
  
        private static boolean esRespuestaValida(String respuesta) {  
            return respuesta.equals(Objects.toString("s")) || respuesta.equals(Objects.toString("n"));  
        }  
    }  
}
```

Programa fuente: Clase que implementa en Java el TDA Cola Medieval

https://github.com/MegaGone/udv/blob/develop/estructuras/parcial_II/Implementacion.java

```
PS C:\repos\udv\estructuras\parcial_II> & 'C:\java\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\jimmy\AppData\Roaming\Code\User\workspaceStorage\93817222e8c83e32ba0d9993b9e4402\redhat_java\jdt_ws\jdt.ls-jav
a-project\bin' 'parcial_II\Implementacion'
Ingrese el nombre del personaje 01:
Sir Legolas
¿Es noble? (Si/No):
Si
Ingrese el nombre del personaje 11:
Sir Aragorn
¿Es noble? (Si/No):
Si
Ingrese el nombre del personaje 21:
Sir Gandalf
¿Es noble? (Si/No):
Si
Ingrese el nombre del personaje 31:
Elfo
¿Es noble? (Si/No):
No
Ingrese el nombre del personaje 41:
Orco
¿Es noble? (Si/No):
No
*****
EL PRIMERO EN LA COLA ES: Sir Legolas
ELIMINANDO PRIMERO EN LA COLA (Sir Legolas)
EL PRIMERO EN LA COLA ACTUALIZADA ES: Sir Aragorn
NUMERO DE NOBLES EN LA COLA: 2
NUMERO DE PLEBEYOS EN LA COLA: 2
¿ESTA VACIA LA COLA? false
*****
PS C:\repos\udv\estructuras\parcial_II>
```